

13. Machine Learning: Neural Networks

Ack: Berkeley AI course

Machine Learning

- Up until now: how to use a model to make optimal decisions
- Machine learning: how to acquire a model from data / experience
 - Learning parameters (e.g. probabilities)
 - Learning structure (e.g. BN graphs)
 - Learning hidden concepts (e.g. clustering, neural nets)

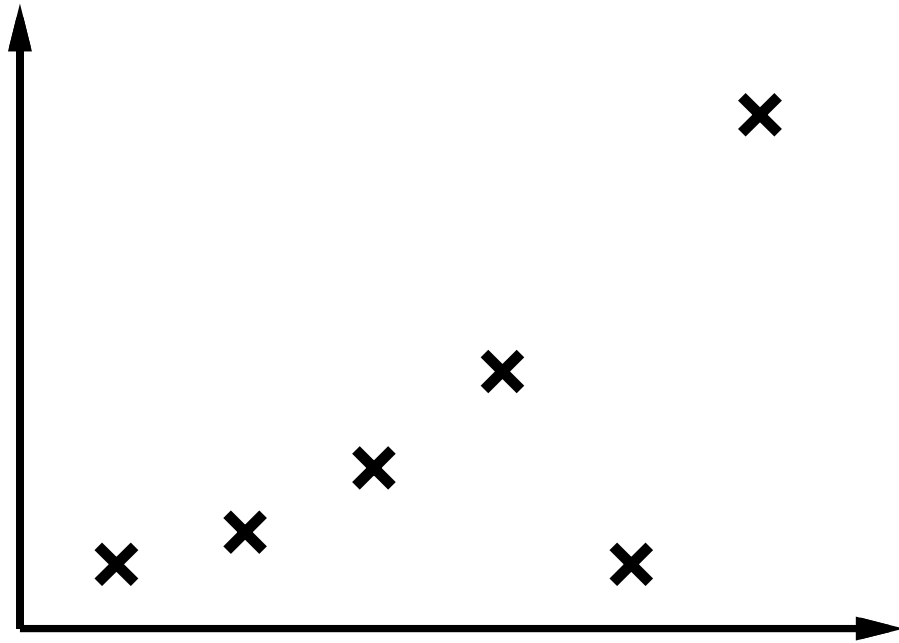
Types of Learning

- *Supervised learning*: correct answers for each training instance
- *Reinforcement learning*: reward sequence, no correct answers
- *Unsupervised learning*: correct answers not given
 - “just make sense of the data”

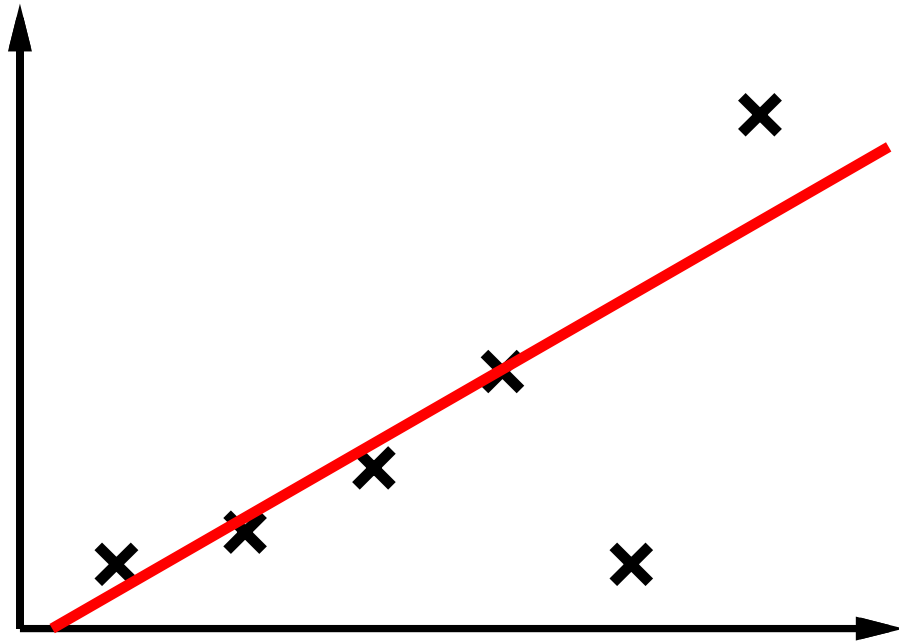
Supervised learning

- To learn an unknown **target function** f
- Training data: a **training set** of **labeled examples** (x_j, y_j) where $y_j = f(x_j)$
 - E.g., x_j is an image, $f(x_j)$ is the label “giraffe”
 - E.g., x_j is an email, $f(x_j)$ is the label “spam”
- Learning result: **hypothesis** h that is “close” to f , i.e., predicts well on unseen examples (“**test set**”)
- Many possible hypothesis families for h
 - Linear models, logistic regression, neural networks, decision trees, examples (nearest-neighbor), grammars, kernelized separators, etc etc
- **Regression** = learning f with real-valued output value
- **Classification** = learning f with discrete output value

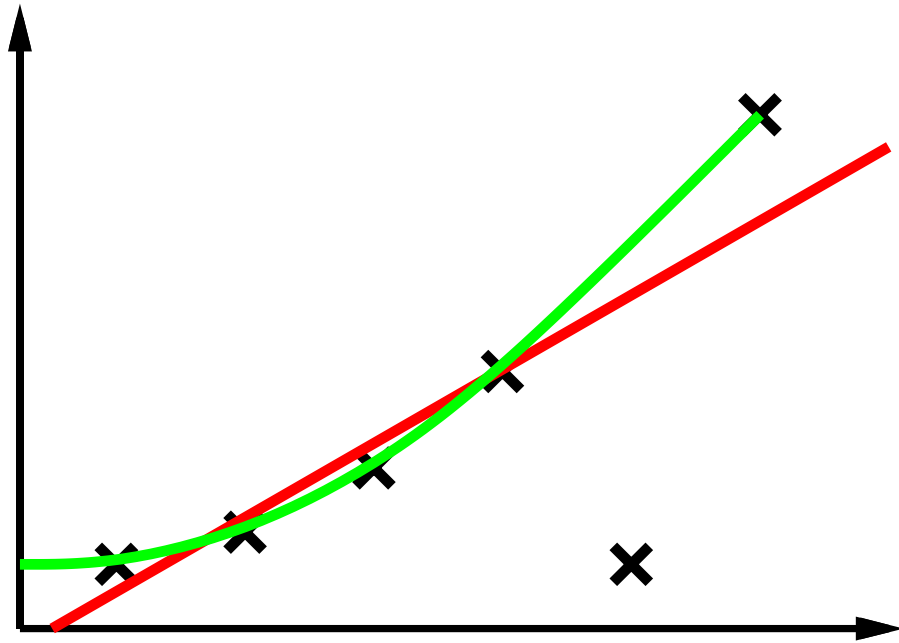
Regression example: Curve fitting



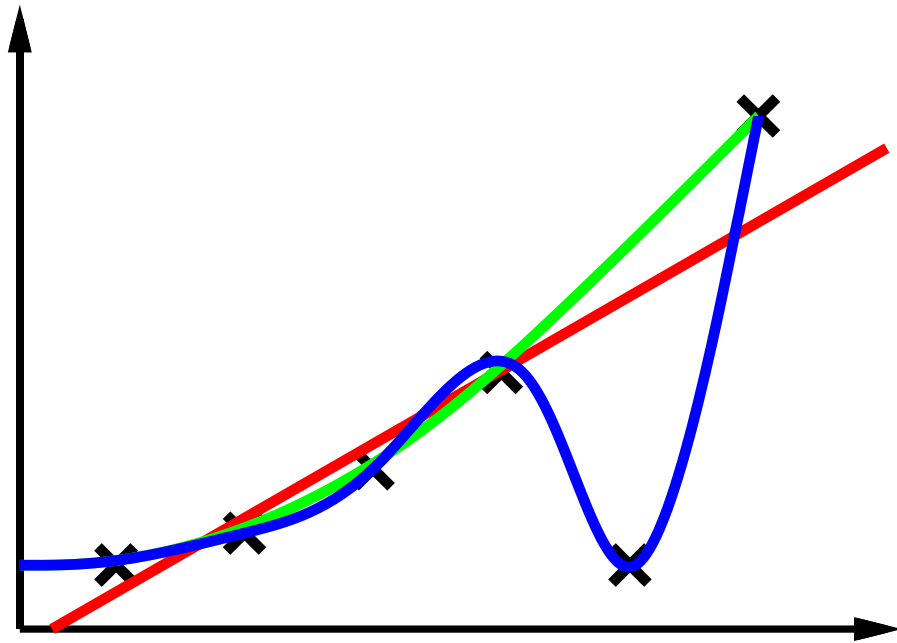
Regression example: Curve fitting



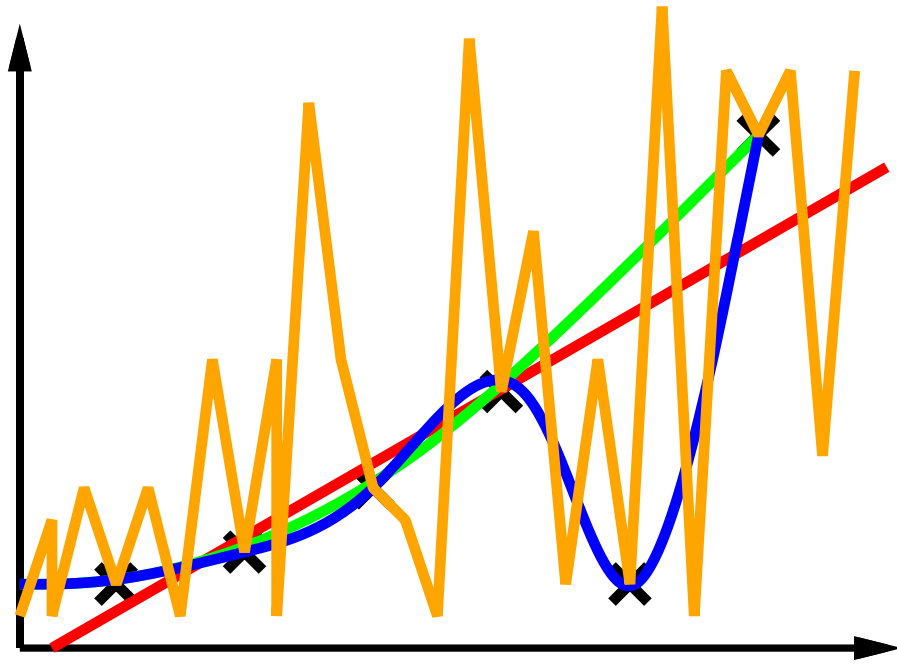
Regression example: Curve fitting



Regression example: Curve fitting



Regression example: Curve fitting



Ockham's razor: prefer the simplest hypothesis consistent with data

Classification example: Spam Filter

- Classification
 - Input: an email
 - Output: spam/ham
- Setup:
 - Get a large collection of example emails, each labeled “spam” or “ham” (by hand)
 - Learn to predict labels of new incoming emails
 - Classifiers reject 200 billion spam emails per day
- Features: The attributes used to make the ham / spam decision
 - Words: FREE!
 - Text Patterns: \$dd, CAPS
 - Non-text: SenderInContacts, AnchorLinkMismatch
 - ...



Dear Sir.

First, I must solicit your confidence in this transaction, this is by virtue of its nature as being utterly confidential and top secret. ...



TO BE REMOVED FROM FUTURE MAILINGS, SIMPLY REPLY TO THIS MESSAGE AND PUT "REMOVE" IN THE SUBJECT.

99 MILLION EMAIL ADDRESSES
FOR ONLY \$99



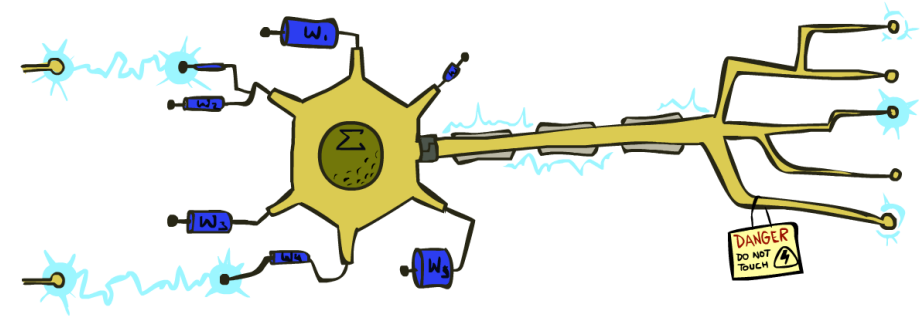
Ok, I know this is blatantly OT but I'm beginning to go insane. Had an old Dell Dimension XPS sitting in the corner and decided to put it to use, I know it was working pre being stuck in the corner, but when I plugged it in, hit the power nothing happened.

Other Classification Tasks

- Medical diagnosis
 - input: symptoms
 - output: disease
- Automatic essay grading
 - input: document
 - output: grades
- Fraud detection
 - input: account activity
 - output: fraud / no fraud
- ... many more

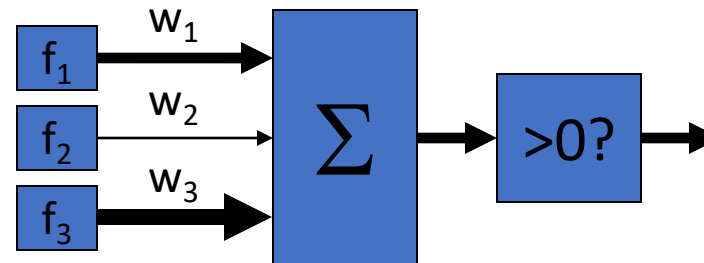
Recap: Linear Classifiers

- Inputs are feature values
- Each feature has a weight
- Sum is the activation



$$\text{activation}_w(x) = \sum_i w_i \cdot f_i(x) = w \cdot f(x)$$

- If the activation is:
 - Positive, output +1
 - Negative, output -1



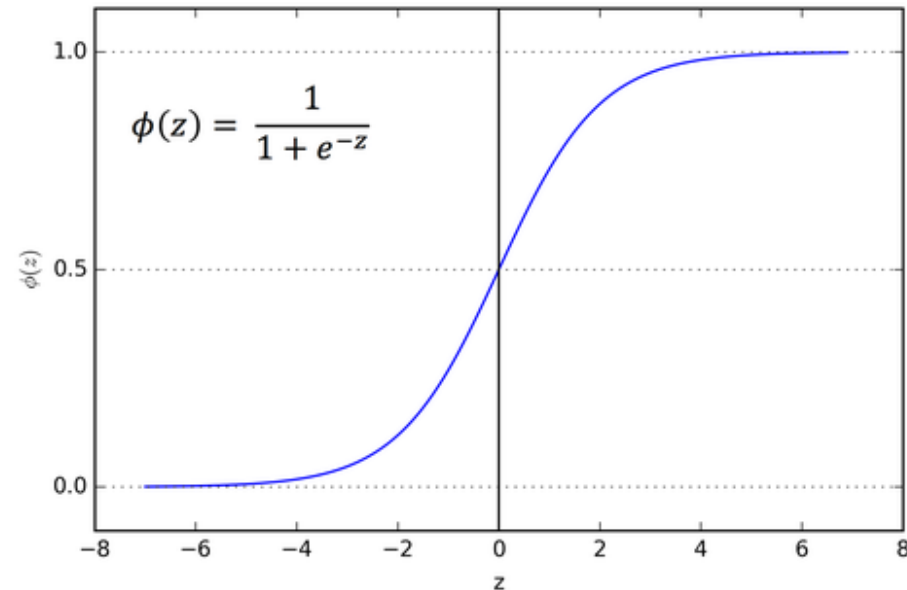
How to get probabilistic decisions?

$$z = w \cdot f(x)$$

- Activation:
- If $z = w \cdot f(x)$ very positive $\rightarrow$ want probability going to 1
- If $z = w \cdot f(x)$ very negative $\rightarrow$ want probability going to 0

- Sigmoid function

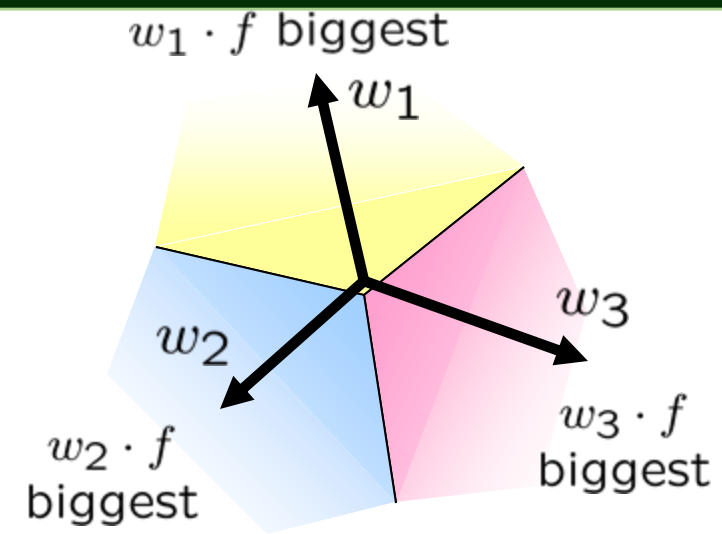
$$\phi(z) = \frac{1}{1 + e^{-z}}$$



Multiclass Logistic Regression

- Multi-class linear classification

- A weight vector for each class: w_y
- Score (activation) of a class y : $z = w_y \cdot f(x)$
- Prediction w/highest score wins: $y = \arg \max_y w_y \cdot f(x)$

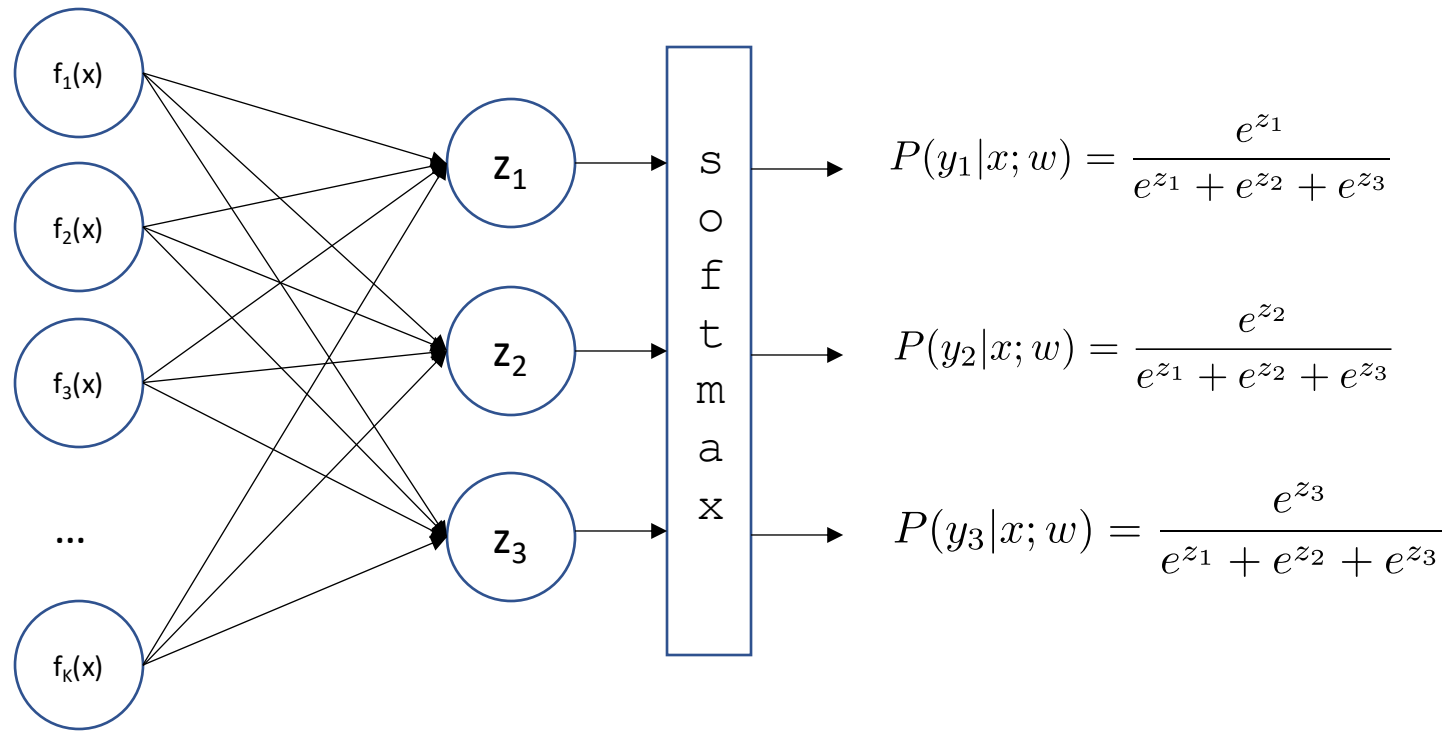


- How to make the scores into probabilities?

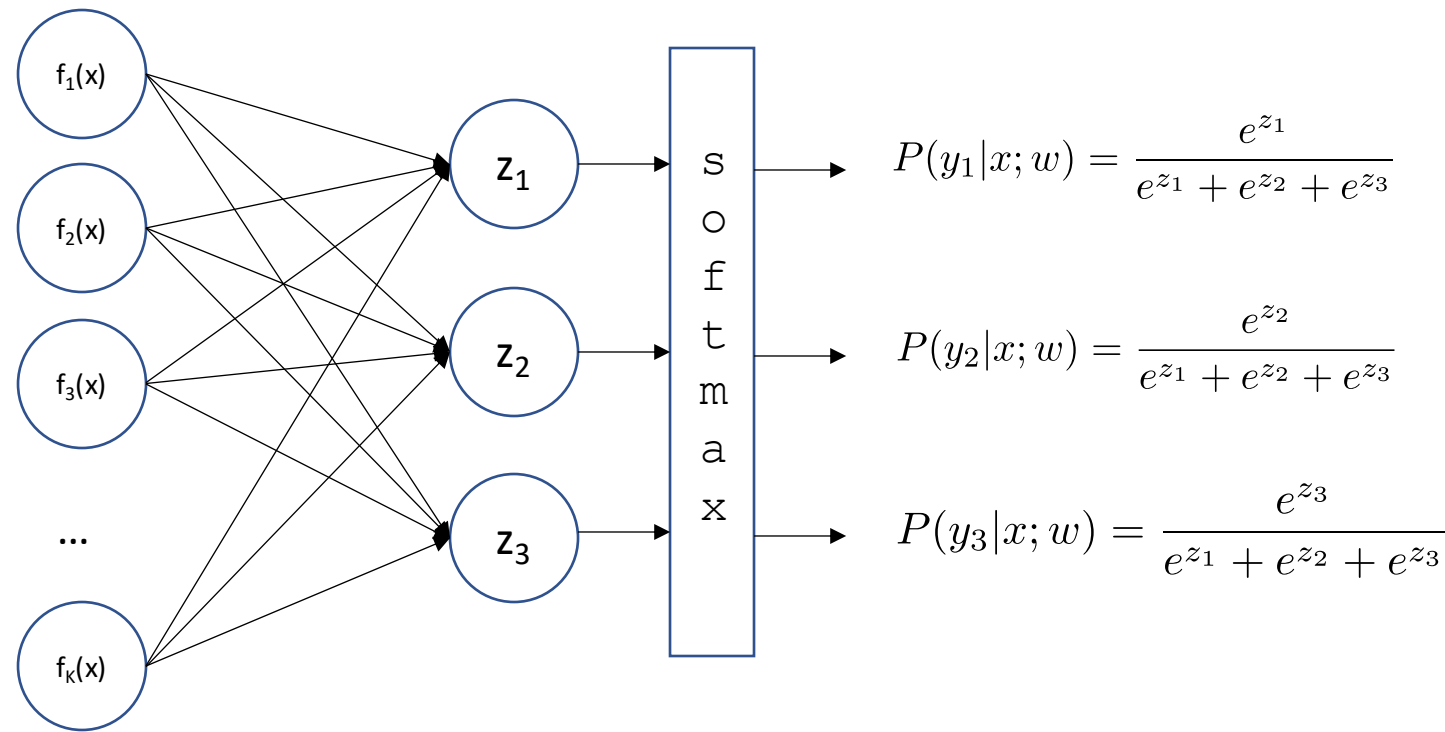
$$\underbrace{z_1, z_2, z_3}_{\text{original activations}} \rightarrow \underbrace{\frac{e^{z_1}}{e^{z_1} + e^{z_2} + e^{z_3}}, \frac{e^{z_2}}{e^{z_1} + e^{z_2} + e^{z_3}}, \frac{e^{z_3}}{e^{z_1} + e^{z_2} + e^{z_3}}}_{\text{softmax activations}}$$

Multiclass Logistic Regression

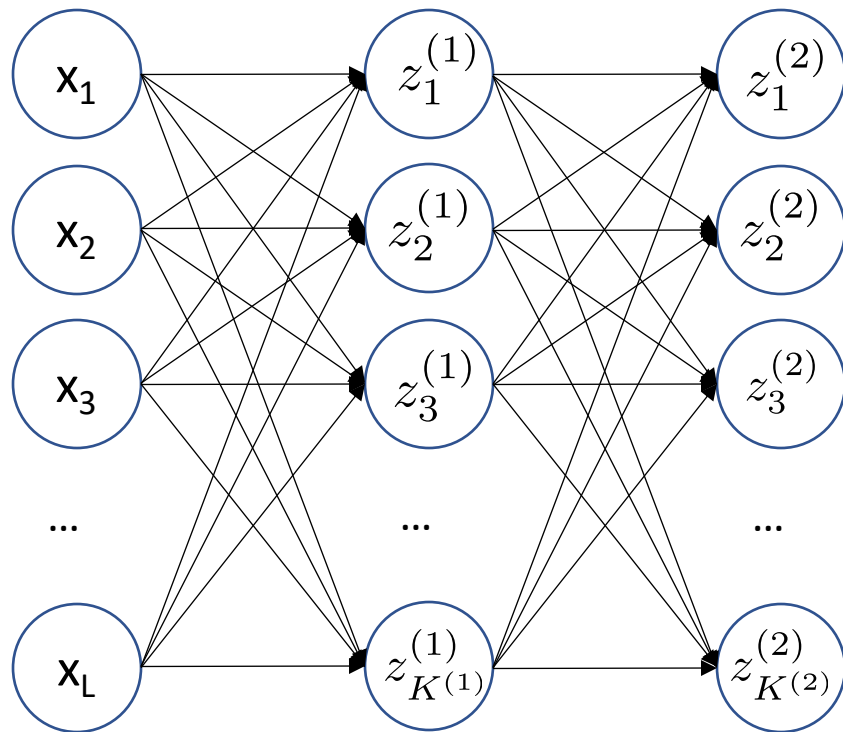
- = special case of neural network



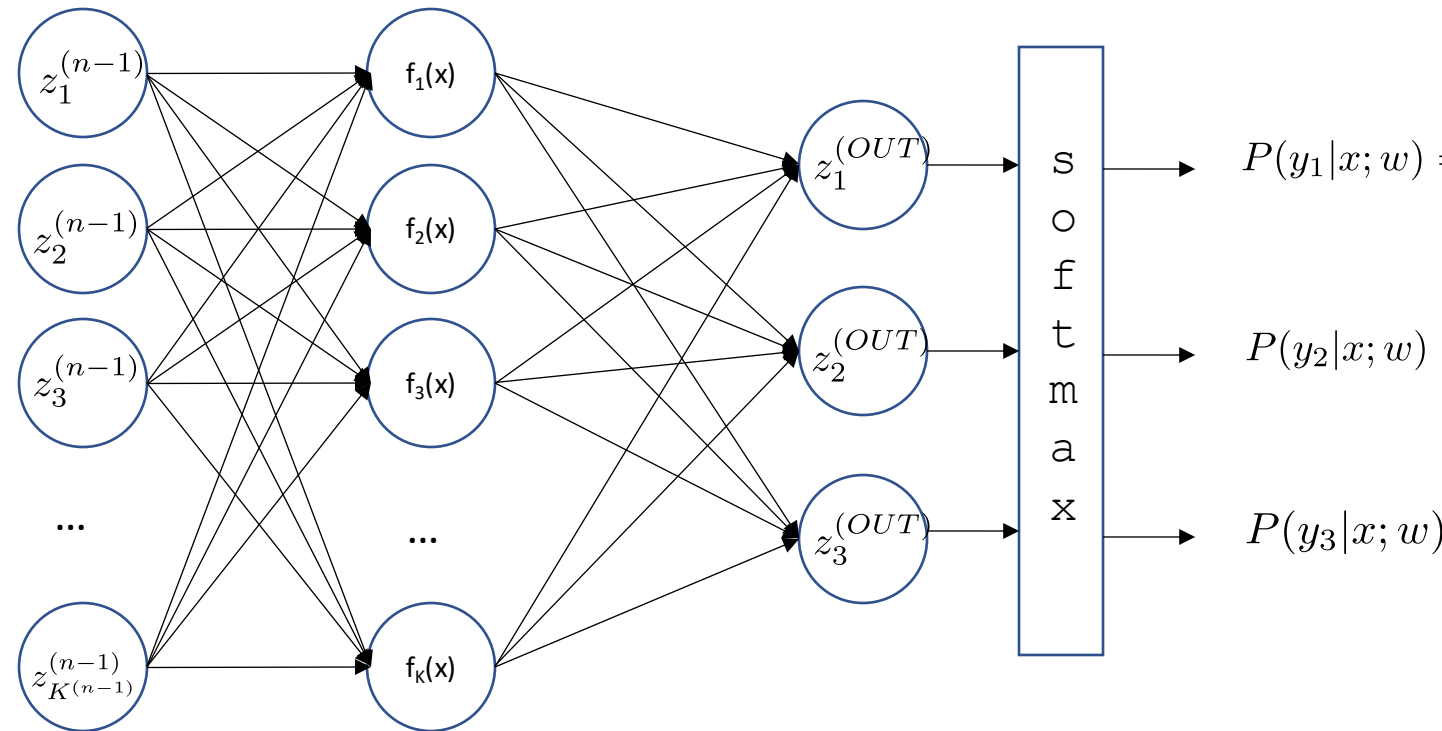
Deep Neural Network = Also learn the features!



Deep Neural Network = Also learn the features!



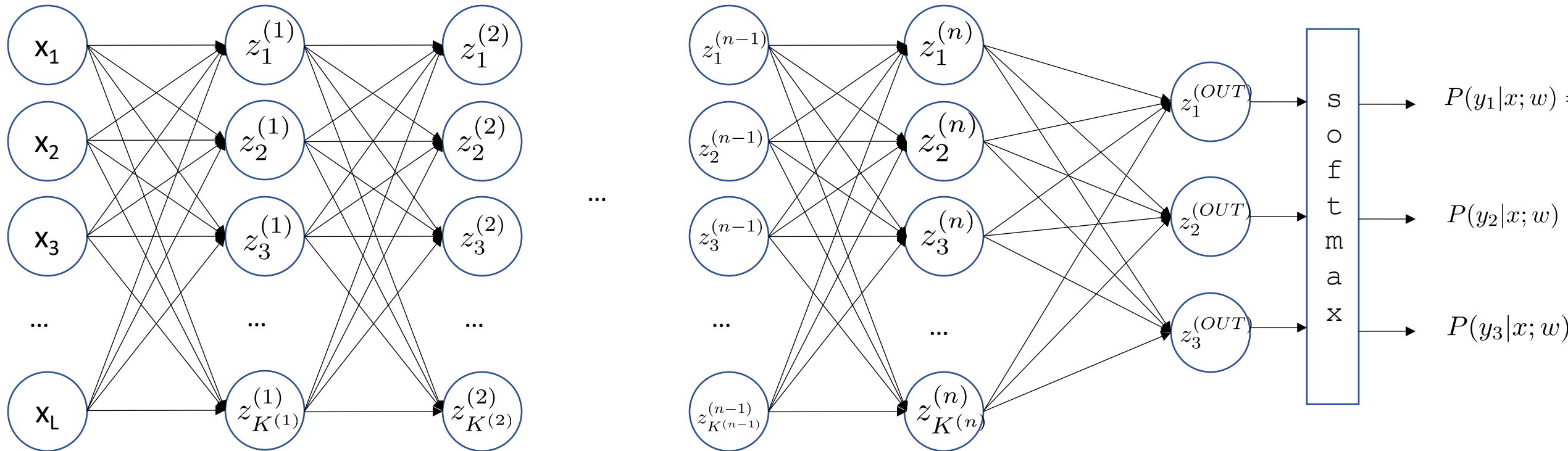
...



$$z_i^{(k)} = g\left(\sum_j W_{i,j}^{(k-1,k)} z_j^{(k-1)}\right)$$

g = nonlinear activation function

Deep Neural Network = Also learn the features!

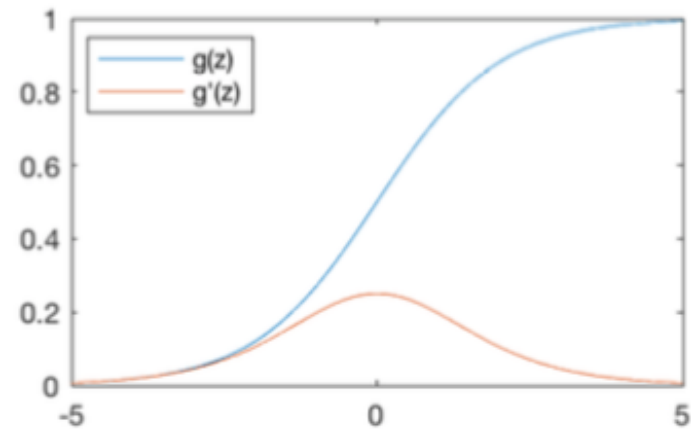


$$z_i^{(k)} = g\left(\sum_j W_{i,j}^{(k-1,k)} z_j^{(k-1)}\right)$$

g = nonlinear activation function

Common Activation Functions

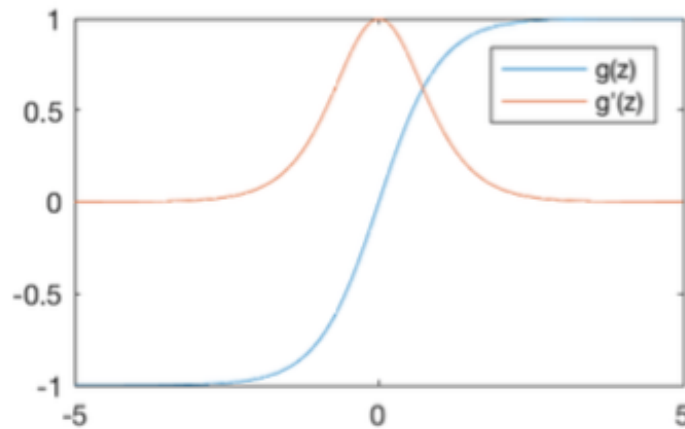
Sigmoid Function



$$g(z) = \frac{1}{1 + e^{-z}}$$

$$g'(z) = g(z)(1 - g(z))$$

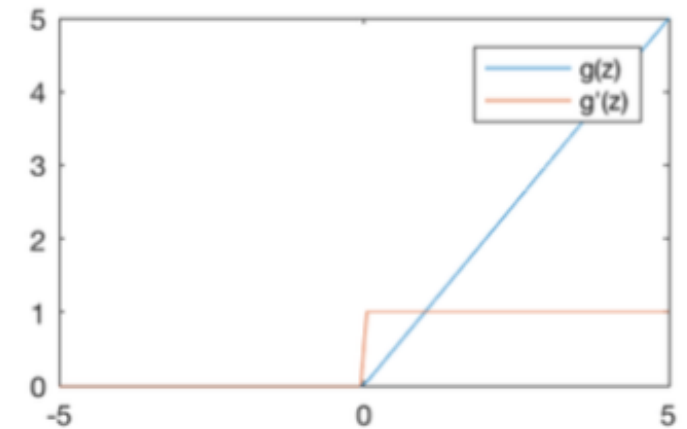
Hyperbolic Tangent



$$g(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}$$

$$g'(z) = 1 - g(z)^2$$

Rectified Linear Unit (ReLU)



$$g(z) = \max(0, z)$$

$$g'(z) = \begin{cases} 1, & z > 0 \\ 0, & \text{otherwise} \end{cases}$$

Deep Neural Network: Also Learn the Features!

- Training the deep neural network is just like logistic regression:

$$\max_w ll(w) = \max_w \sum_i \log P(y^{(i)} | x^{(i)}; w)$$

just w tends to be a much, much larger vector 😊

→ just run gradient ascent

+ stop when log likelihood of hold-out data starts to decrease

Neural Networks Properties

- Theorem (Universal Function Approximators). A two-layer neural network with a sufficient number of neurons can approximate any continuous function to any desired accuracy.
- Practical considerations
 - Can be seen as learning the features
 - Large number of neurons
 - Danger for overfitting
 - (hence early stopping!)

How about computing all the derivatives?

- Derivatives tables:

$$\frac{d}{dx}(a) = 0$$

$$\frac{d}{dx}(x) = 1$$

$$\frac{d}{dx}(au) = a \frac{du}{dx}$$

$$\frac{d}{dx}(u + v - w) = \frac{du}{dx} + \frac{dv}{dx} - \frac{dw}{dx}$$

$$\frac{d}{dx}(uv) = u \frac{dv}{dx} + v \frac{du}{dx}$$

$$\frac{d}{dx}\left(\frac{u}{v}\right) = \frac{1}{v} \frac{du}{dx} - \frac{u}{v^2} \frac{dv}{dx}$$

$$\frac{d}{dx}(u^n) = nu^{n-1} \frac{du}{dx}$$

$$\frac{d}{dx}(\sqrt{u}) = \frac{1}{2\sqrt{u}} \frac{du}{dx}$$

$$\frac{d}{dx}\left(\frac{1}{u}\right) = -\frac{1}{u^2} \frac{du}{dx}$$

$$\frac{d}{dx}\left(\frac{1}{u^n}\right) = -\frac{n}{u^{n+1}} \frac{du}{dx}$$

$$\frac{d}{dx}[f(u)] = \frac{d}{du}[f(u)] \frac{du}{dx}$$

$$\frac{d}{dx}[\ln u] = \frac{d}{dx}[\log_e u] = \frac{1}{u} \frac{du}{dx}$$

$$\frac{d}{dx}[\log_a u] = \log_a e \frac{1}{u} \frac{du}{dx}$$

$$\frac{d}{dx}e^u = e^u \frac{du}{dx}$$

$$\frac{d}{dx}a^u = a^u \ln a \frac{du}{dx}$$

$$\frac{d}{dx}(u^v) = vu^{v-1} \frac{du}{dx} + \ln u \cdot u^v \frac{dv}{dx}$$

$$\frac{d}{dx} \sin u = \cos u \frac{du}{dx}$$

$$\frac{d}{dx} \cos u = -\sin u \frac{du}{dx}$$

$$\frac{d}{dx} \tan u = \sec^2 u \frac{du}{dx}$$

$$\frac{d}{dx} \cot u = -\csc^2 u \frac{du}{dx}$$

$$\frac{d}{dx} \sec u = \sec u \tan u \frac{du}{dx}$$

$$\frac{d}{dx} \csc u = -\csc u \cot u \frac{du}{dx}$$

How about computing all the derivatives?

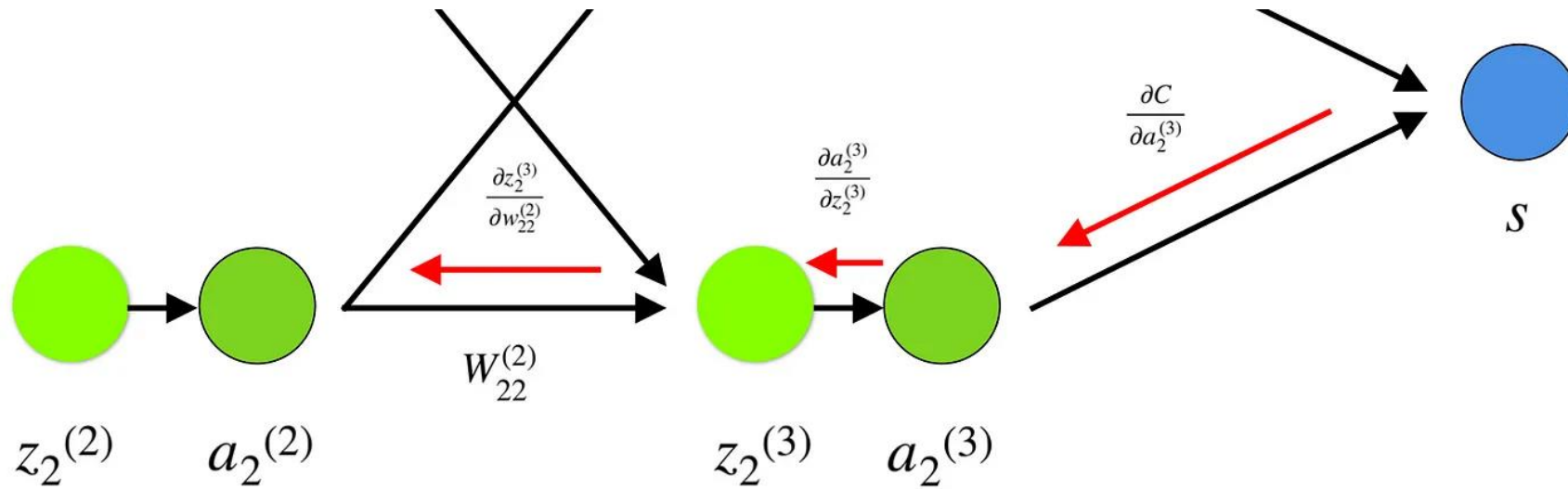
- But neural net f is never one of those?
 - No problem: CHAIN RULE:

If $f(x) = g(h(x))$

Then $f'(x) = g'(h(x))h'(x)$

→ Derivatives can be computed by following well-defined procedures

Backpropagation



$$\frac{\partial C}{\partial w_{22}^{(2)}} = \frac{\partial C}{\partial z_2^{(3)}} \cdot \frac{\partial z_2^{(3)}}{\partial w_{22}^{(2)}} = \frac{\partial C}{\partial a_2^{(3)}} \cdot \frac{\partial a_2^{(3)}}{\partial z_2^{(3)}} \cdot a_2^{(2)} = \frac{\partial C}{\partial a_2^{(3)}} \cdot f'(z_2^{(3)}) \cdot a_2^{(2)}$$

Pseudocode for NN (backprop)

```
//Propagate the errors backward through the network
for every node in the output layer
    calculate the error signal
end
for all hidden layers
    for every node in the layer
        1. Calculate the derivatives at the node (wts and biases)
        2. Update each node's weight in the network
    end
end

//Calculate Global Error
Calculate the Error Function
end // for every data point
end /while...do
```

$$\frac{\partial C}{\partial w_{jk}^l} = \frac{\partial C}{\partial z_j^l} \frac{\partial z_j^l}{\partial w_{jk}^l} \quad \text{chain rule}$$

$$z_j^l = \sum_{k=1}^m w_{jk}^l a_k^{l-1} + b_j^l \quad \text{by definition}$$

m – number of neurons in $l-1$ layer

$$\frac{\partial z_j^l}{\partial w_{jk}^l} = a_k^{l-1} \quad \text{by differentiation (calculating derivative)}$$

$$\frac{\partial C}{\partial w_{jk}^l} = \frac{\partial C}{\partial z_j^l} a_k^{l-1} \quad \text{final value}$$

while (termination condition not met)

$$w := w - \epsilon \frac{\partial C}{\partial w}$$

$$b := b - \epsilon \frac{\partial C}{\partial b}$$

end

Training of multi-layer networks

- Find network weights to minimize the error between true and estimated labels of training examples:

$$E(\mathbf{w}) = \sum_{j=1}^N \left(y_j - f_{\mathbf{w}}(\mathbf{x}_j) \right)^2$$

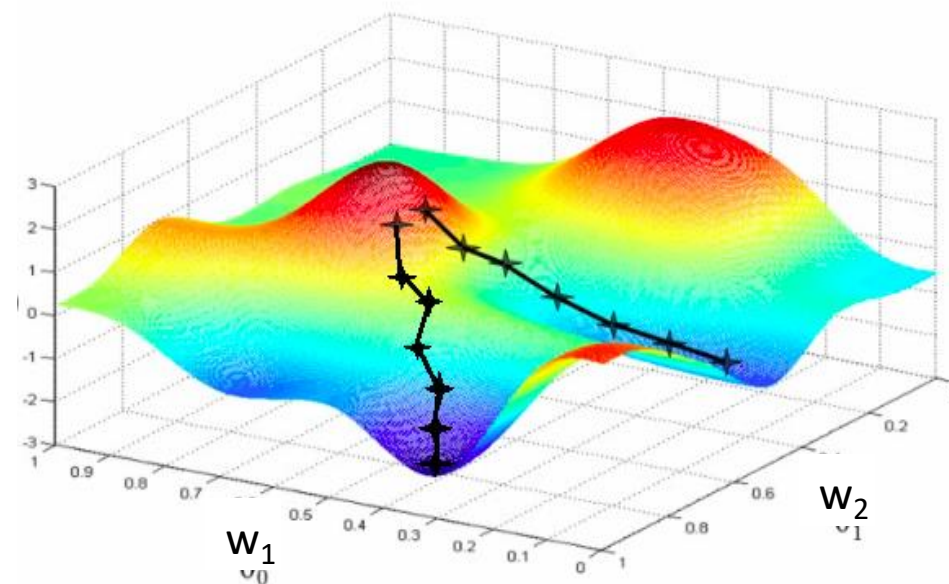
- Update weights by **gradient descent**: $\mathbf{w} \leftarrow \mathbf{w} - \alpha \frac{\partial E}{\partial \mathbf{w}}$
- Back-propagation**: gradients are computed in the direction from output to input layers and combined using chain rule
- Stochastic gradient descent**: compute the weight update w.r.t. one training example (or a small batch of examples) at a time, cycle through training examples in random order in multiple epochs

Training of multi-layer networks

- Find network weights to minimize the error between true and estimated labels of training examples:

$$E(\mathbf{w}) = \sum_{j=1}^N (y_j - f_{\mathbf{w}}(\mathbf{x}_j))^2$$

- Update weights by **gradient descent**: $\mathbf{w} \leftarrow \mathbf{w} - \alpha \frac{\partial E}{\partial \mathbf{w}}$



Neural networks: Pros and cons

- **Pros**
 - Flexible and general function approximation framework
 - Can build extremely powerful models by adding more layers
- **Cons**
 - Hard to analyze theoretically (e.g., training is prone to local optima)
 - Huge amount of training data, computing power may be required to get good performance
 - The space of implementation choices is huge (network architectures, parameters)